

Recommender Systems

COMPSCI 753

Recap: Collaborative Filtering (CF)

- Use the similarity to weight the ratings of items from other user u_k

$$\hat{r}_{ij} = \frac{\sum_{u_k \in U_j} w_{ik} r_{kj}}{\sum_{u_k \in U_j} |w_{ik}|}$$

w_{ik} - similarity between two users

U_j - set of most similar users who rated item p_j

When predicting the rating, we can either consider
1) all the users or 2) only k most similar users

Modeling Bias

- Use the similarity to weight the ratings of items from other user u_k

$$\hat{r}_{ij} = b_{ij} + \frac{\sum_{u_k \in U_j} w_{ik}(r_{kj} - b_{kj})}{\sum_{u_k \in U_j} |w_{ik}|}$$

$$b_{ij} = b_g + b_i^{(user)} + b_j^{(item)}$$

b_g - avg ratings over all transactions

$b_i^{(user)}$ - avg ratings of user i - b_g

$b_j^{(item)}$ - avg ratings of item j - b_g

Problem: The similarity is based on arbitrary metric. Can we learn the similarities?

Solution: View the recommendation problem as an interpolation problem for missing values and learn the interpolation weights

Learning the interpolation weights

- Learn the interpolation weights α_{ik} instead of arbitrary similarity

$$\hat{r}_{ij} = b_{ij} + \sum_{u_k \in U_j} \alpha_{ik} (r_{kj} - b_{kj})$$

- α_{ik} - the interpolation weight that models the impact of user k to user i .
- Note that
 - α_{ik} may not sum up to 1: $\sum_{u_k \in U_j} \alpha_{ik} \neq 1$
 - α_{ik} may not be symmetric: $\alpha_{ik} \neq \alpha_{ki}$

Learning the interpolation weights

- Our prediction model:

$$\hat{r}_{ij} = b_{ij} + \sum_{u_k \in U_j} \alpha_{ik} (r_{kj} - b_{kj})$$

- Question: How to learn the value of α_{ik} ?**

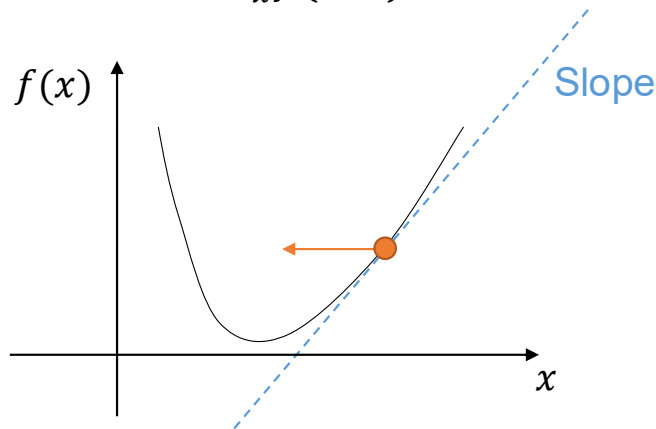
- Idea - minimize the difference between predicted and real ratings!

$$SSE = \sum_{(i,j) \in R} (\hat{r}_{ij} - r_{ij})^2$$

- Merit – directly optimize the RMSE metric!
- The weight α_{ik} is then learned based on all the items purchased by both users.
- The same approach can be applied to item-based method!

Learning the interpolation weights

- How to minimize a function $f(x)$?
 - The function describes a surface, e.g., a mountain
 - At time t , find the steepest direction to go down
 1. Take derivatives of the function $\nabla_x f$
 2. Gradient $\nabla_x f(x^{(t)})$ – the value of $\nabla_x f$ at point $x^{(t)}$
 3. Go along the reverse direction of gradient
 4. $x^{(t+1)} \leftarrow x^{(t)} - \nabla_x f(x^{(t)})$

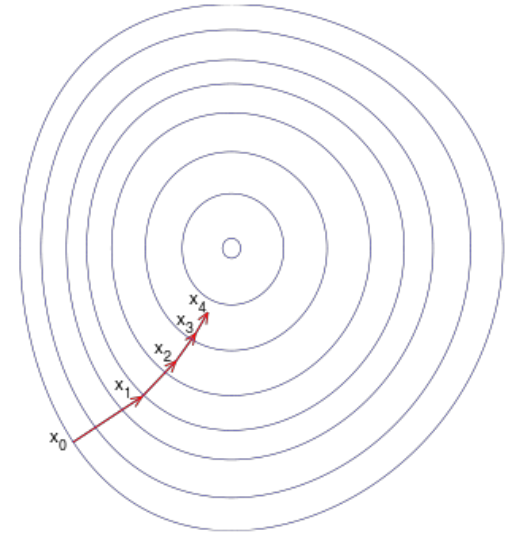
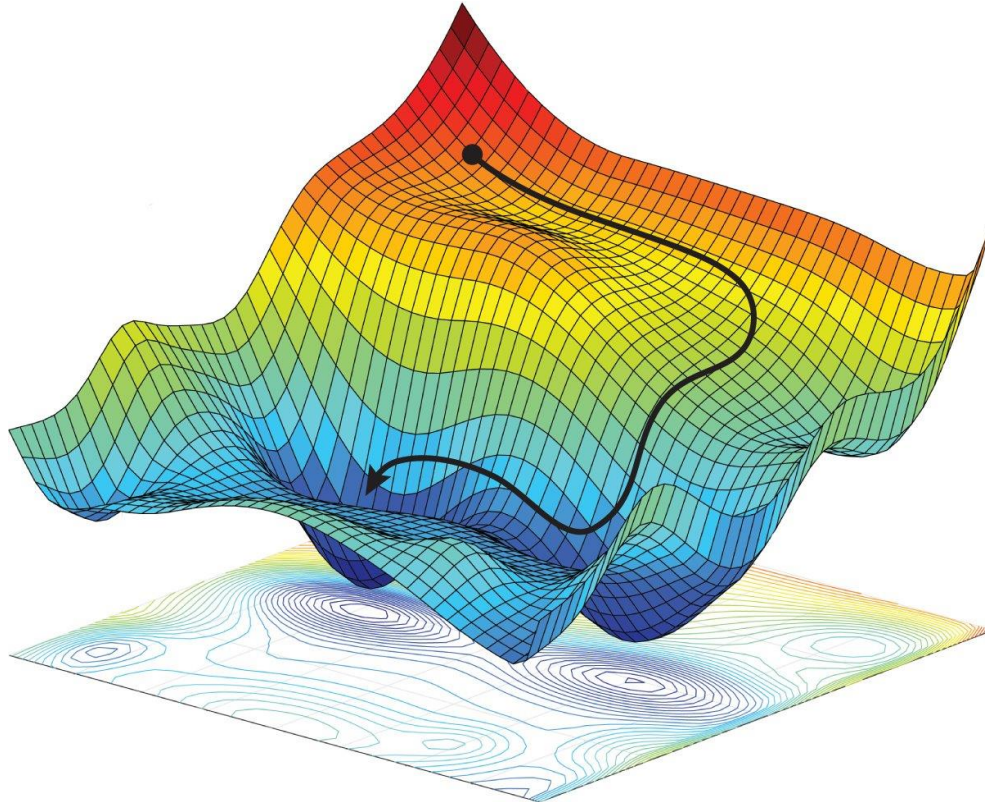


Side notes:

- The sign of the slope gives the direction towards a larger value of $f(x)$
- Go along the opposite direction of the gradient will get to lower value of $f(x)$

Gradient descent

- View from top and get a contour plot



The (minus) gradient points us to the local minima

Learning the interpolation weights

- The optimization problem:

$$\min_{\alpha} \mathcal{L}(\alpha) = \min_{\alpha} \sum_{(i,j) \in R} \left(b_{ij} + \sum_{u_k \in U_j} \alpha_{ik} (r_{kj} - b_{kj}) - r_{ij} \right)^2$$

- **Gradient descent** – iterative learning method:

- $\alpha^{(t+1)} \leftarrow \alpha^{(t)} - \eta \nabla_{\alpha} \mathcal{L}^{(t)}$, where η is called **learning rate**.

$$\alpha = \begin{bmatrix} \alpha_{11} & \alpha_{12} \\ \alpha_{21} & \alpha_{22} \end{bmatrix}$$

- How to take derivative $\nabla_{\alpha} \mathcal{L}^{(t)}$ w.r.t. to the matrix α

- take partial derivative w.r.t. each α_{ik} - $\nabla_{\alpha_{ik}} \mathcal{L}^{(t)} = \frac{\partial \mathcal{L}^{(t)}}{\partial \alpha_{ik}}$

$$\nabla_{\alpha} \mathcal{L}^{(t)} = \begin{bmatrix} \frac{\partial \mathcal{L}^{(t)}}{\partial \alpha_{11}} & \frac{\partial \mathcal{L}^{(t)}}{\partial \alpha_{12}} \\ \frac{\partial \mathcal{L}^{(t)}}{\partial \alpha_{21}} & \frac{\partial \mathcal{L}^{(t)}}{\partial \alpha_{22}} \end{bmatrix}$$

- $\frac{\partial \mathcal{L}^{(t)}}{\partial \alpha_{ik}} = 2 \underbrace{\sum_{j \in R[i,:]} [b_{ij} + \sum_{u_k \in U_j} \alpha_{ik}^{(t)} (r_{kj} - b_{kj}) - r_{ij}]}_{\text{Sum over all items rated by user } i} (r_{kj} - b_{kj})$

Sum over all items rated by user i

Stochastic Gradient Descent

- Idea – compute gradient w.r.t. each individual rating, denoted by $\nabla_{\alpha} \mathcal{L}_{ij}^{(t)}$ and make a step to update the parameters

$$\frac{\partial \mathcal{L}^{(t)}}{\partial \alpha_{ik}} = \sum_{j \in R[i,:]} \underbrace{2[b_{ij} + \sum_{u_k \in U_j} \alpha_{ik}^{(t)}(r_{kj} - b_{kj}) - r_{ij}]}_{\text{The gradient w.r.t. one rating } r_{ij}} (r_{kj} - b_{kj})$$

The gradient w.r.t. one rating r_{ij}

- For each rating r_{ij} , update $\alpha_{ik}^{(t+1)} \leftarrow \alpha_{ik}^{(t)} - \eta \nabla_{\alpha_{ik}} \mathcal{L}_{ij}^{(t)}$
- SGD runs several passes (epochs) on the full training dataset

- GD:** $\alpha^{(t+1)} \leftarrow \alpha^{(t)} - \eta [\sum_{r_{ij} \in R} \nabla_{\alpha} \mathcal{L}_{ij}^{(t)}]$

- SGD:** $\alpha^{(t+1)} \leftarrow \alpha^{(t)} - \eta \nabla_{\alpha} \mathcal{L}_{ij}^{(t)}$

SGD Needs more steps to converge
but each step is efficient!

SGD vs. GD

- Convergence of **GD** vs. **SGD**

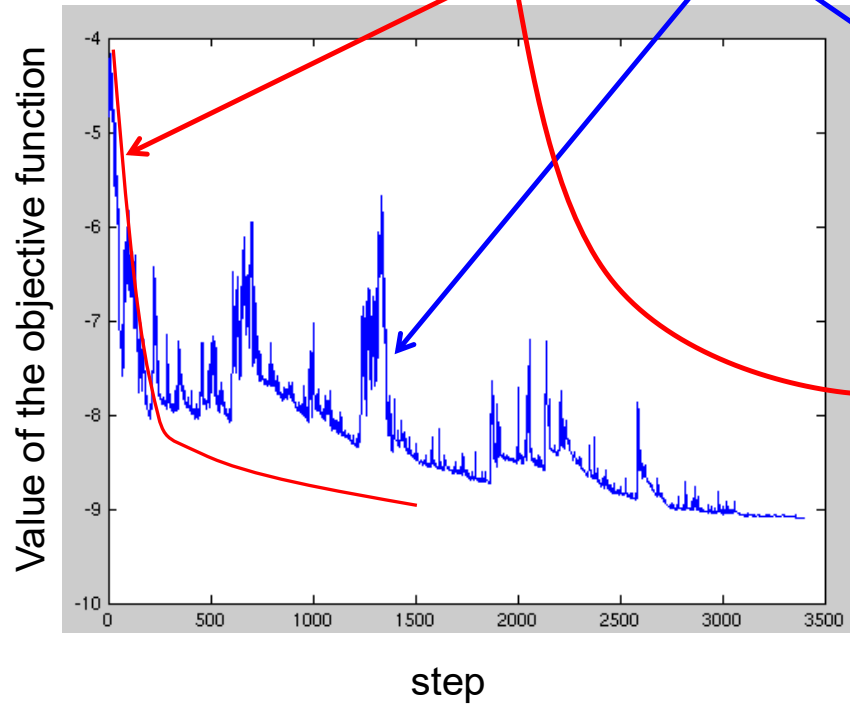
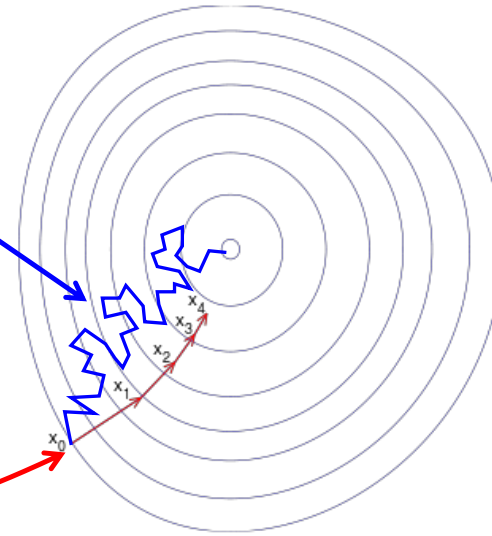


Figure from mmds.org



SGD may detour but often goes faster towards the optimal